```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

print("OpenCV Version:", cv2.__version__)
print("Libraries imported successfully!")
```

```
OpenCV Version: 5.0.0
Libraries imported successfully!
```

```
uploaded = files.upload()

video_path = list(uploaded.keys())[0]

print("Uploaded video:", video_path)
```

Choose Files 16011564_…_60fps.mp4

16011564_1080_1920_60fps.mp4(video/mp4) - 9281155 bytes, last modified: 9/17/2026 - 100% done

```
Saving 16011564_1080_1920_60fps.mp4 to 16011564_1080_1920_60fps.mp4
Uploaded video: 16011564_1080_1920_60fps.mp4
```

```
cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Could not open video.")
else:
    print("Video opened successfully!")

fps = cap.get(cv2.CAP_PROP_FPS)
frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

print("FPS:", fps)
print("Total Frames:", frame_count)
print("Width:", width)
print("Height:", height)
```

```
Video opened successfully!
FPS: 59.94005994005994
Total Frames: 960
Width: 1080
Height: 1920
```

```
cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

ret, frame1 = cap.read()
ret2, frame2 = cap.read()

if not ret or not ret2:
    print("Could not read frames.")
else:
    print("Two consecutive frames loaded successfully.")

frame1_rgb = cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB)
frame2_rgb = cv2.cvtColor(frame2, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(frame1_rgb)
plt.title("Frame 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(frame2_rgb)
plt.title("Frame 2")
plt.axis("off")

plt.show()
```

```
Two consecutive frames loaded successfully.
```



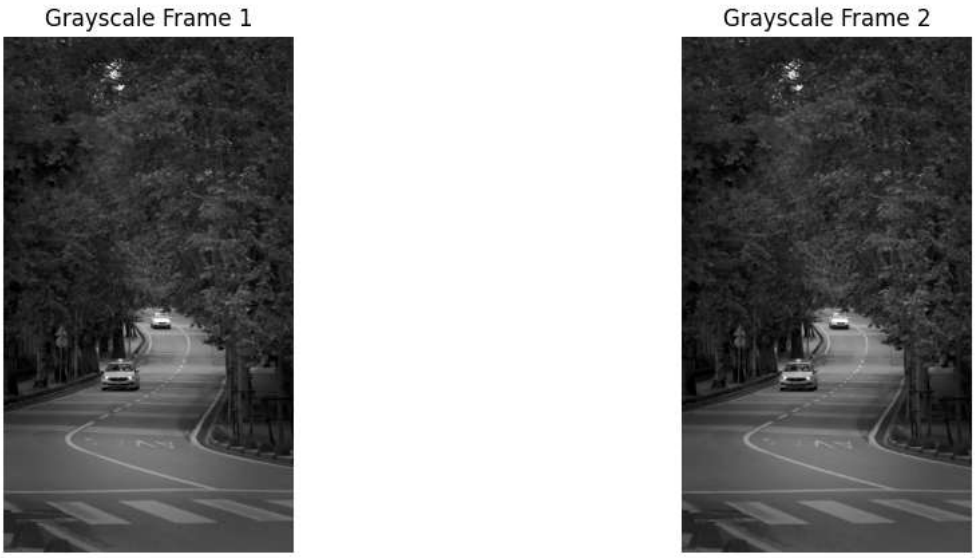

```
gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(gray1, cmap='gray')
plt.title("Grayscale Frame 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(gray2, cmap='gray')
plt.title("Grayscale Frame 2")
plt.axis("off")

plt.show()
```




```
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

p0 = cv2.goodFeaturesToTrack(
    gray1,
    mask=None,
```

```
    **feature_params
)

if p0 is not None:
    print("Number of feature points detected:", len(p0))
else:
    print("No feature points detected.")
```

```
Number of feature points detected: 59
```

```
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)

p1, status, error = cv2.calcOpticalFlowPyrLK(
    gray1,
    gray2,
    p0,
    None,
    **lk_params
)

if p1 is not None:
    good_new = p1[status == 1]
    good_old = p0[status == 1]

    print("Tracked points:", len(good_new))
else:
    print("Lucas-Kanade optical flow failed.")
```

```
Tracked points: 59
```

```
lk_output = frame2.copy()

for new, old in zip(good_new, good_old):

    x_new, y_new = new.ravel()
    x_old, y_old = old.ravel()

    cv2.arrowedLine(
        lk_output,
        (int(x_old), int(y_old)),
        (int(x_new), int(y_new)),
        (0, 255, 0),
        2,
        tipLength=0.3
    )

    cv2.circle(
        lk_output,
        (int(x_new), int(y_new)),
        4,
        (0, 0, 255),
        -1
    )

plt.figure(figsize=(12, 7))

plt.imshow(cv2.cvtColor(lk_output, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Sparse Optical Flow")
plt.axis("off")
plt.show()
```




```
flow = cv2.calcOpticalFlowFarneback(
    gray1,
    gray2,
    None,
    pyr_scale=0.5,
    levels=3,
    winsize=15,
    iterations=3,
    poly_n=5,
    poly_sigma=1.2,
    flags=0
)

print("Farneback Dense Optical Flow calculated successfully!")
print("Flow shape:", flow.shape)
```

```
Farneback Dense Optical Flow calculated successfully!
Flow shape: (1920, 1080, 2)
```

```
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

print("Minimum motion magnitude:", magnitude.min())
print("Maximum motion magnitude:", magnitude.max())
print("Average motion magnitude:", magnitude.mean())
```

```
Minimum motion magnitude: 1.0723492e-14
Maximum motion magnitude: 5.9648294
Average motion magnitude: 0.3887193
```

```
hsv = np.zeros_like(frame1)

hsv[..., 1] = 255

hsv[..., 0] = angle * 180 / np.pi / 2

hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
```

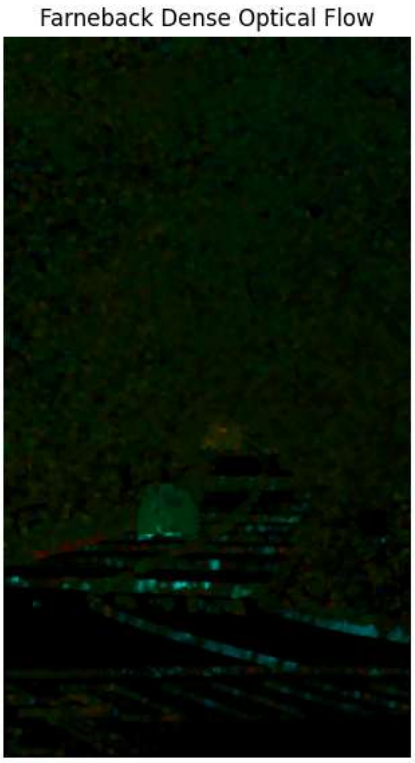
```
)

dense_flow = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2RGB
)

plt.figure(figsize=(12, 7))

plt.imshow(dense_flow)
plt.title("Farneback Dense Optical Flow")
plt.axis("off")

plt.show()
```




```
plt.figure(figsize=(15, 6))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(lk_output, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Sparse Optical Flow")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(dense_flow)
plt.title("Farneback Dense Optical Flow")
plt.axis("off")

plt.show()
```

Lucas-Kanade Sparse Optical Flow



Farneback Dense Optical Flow



```
avg_motion = np.mean(magnitude)
max_motion = np.max(magnitude)

print("Motion Analysis")
print("----------------------------")
print(f"Average Motion Magnitude: {avg_motion:.4f}")
print(f"Maximum Motion Magnitude: {max_motion:.4f}")

if avg_motion < 1:
    print("Observation: Very little motion is detected.")
elif avg_motion < 5:
    print("Observation: Moderate motion is detected.")
else:
    print("Observation: Significant motion is detected.")
```

```
Motion Analysis
----------------------------
Average Motion Magnitude: 0.3887
Maximum Motion Magnitude: 5.9648
Observation: Very little motion is detected.
```

```
avg_motion = np.mean(magnitude)
max_motion = np.max(magnitude)

print("Motion Analysis")
print("----------------------------")
print(f"Average Motion Magnitude: {avg_motion:.4f}")
print(f"Maximum Motion Magnitude: {max_motion:.4f}")

if avg_motion < 1:
    print("Observation: Very little motion is detected.")
elif avg_motion < 5:
    print("Observation: Moderate motion is detected.")
else:
    print("Observation: Significant motion is detected.")
```

```
Motion Analysis
----------------------------
Average Motion Magnitude: 0.3887
Maximum Motion Magnitude: 5.9648
Observation: Very little motion is detected.
```

```
step = 20

plt.figure(figsize=(12, 7))

plt.imshow(cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB))

h, w = gray1.shape
```

```
y, x = np.mgrid[
    step//2:h:step,
    step//2:w:step
].astype(int)

fx = flow[y, x, 0]
fy = flow[y, x, 1]

plt.quiver(
    x,
    y,
    fx,
    fy,
    color='red',
    angles='xy',
    scale_units='xy',
    scale=1
)

plt.title("Farneback Optical Flow Motion Vectors")
plt.axis("off")
plt.show()
```



Feature	Lucas-Kanade Sparse Optical Flow	Farneback Dense Optical Flow
Type	Sparse Optical Flow	Dense Optical Flow
Tracking	Tracks selected feature points	Estimates motion for almost every pixel
Output	Motion vectors at feature points	Dense motion field
Computational Cost	Lower	Higher
Speed	Generally faster	Generally slower
Motion Information	Limited to selected points	Provides motion information across the image
Suitable For	Object tracking and feature tracking	Motion analysis and dynamic scene understanding
Visualization	Arrows and trajectories	Color-coded motion map
Robustness	Effective for distinct features	Useful for detailed motion estimation
Main Advantage	Efficient and relatively fast	Provides comprehensive motion information

Observation

Lucas-Kanade Sparse Optical Flow successfully tracked selected feature points between consecutive video frames. The detected motion vectors represented the movement of important features in the scene.

Farneback Dense Optical Flow estimated motion across the image and produced a color-coded representation of motion direction and magnitude.

Lucas-Kanade requires fewer points and is computationally efficient, making it useful for feature and object tracking. Farneback provides more complete motion information because it estimates motion over a dense set of pixels, but it generally requires more computational resources.

The quality of optical flow estimation depends on factors such as object speed, lighting conditions, camera movement, image texture, and changes between consecutive frames.

Theory Questions

Q1. What is Optical Flow? How is it used in computer vision?

Optical Flow is the apparent motion of objects, surfaces, or the camera between consecutive frames of a video. It is represented using motion vectors that describe the direction and magnitude of movement.

In computer vision, optical flow is used to detect and analyze motion in video sequences. It is useful for applications such as object tracking, video surveillance, autonomous driving, robotics, gesture recognition, sports analytics, and action recognition.

Q2. Explain the working principle of the Lucas-Kanade Optical Flow algorithm.

The Lucas-Kanade algorithm is a Sparse Optical Flow technique that tracks selected feature points between consecutive frames.

It assumes that the pixels within a small neighborhood have similar motion. First, important feature points are detected in the first frame. The algorithm then compares a small window around each feature with the corresponding region in the next frame.

Using image intensity changes and image gradients, it estimates the horizontal and vertical motion of the feature. The resulting displacement is represented using motion vectors or arrows.

Lucas-Kanade is computationally efficient and is commonly used for feature tracking and object tracking.

Q3. What is Dense Optical Flow? How does it differ from Sparse Optical Flow?

Dense Optical Flow estimates motion for a large number of pixels across an image, providing a detailed representation of the motion field.

Sparse Optical Flow estimates motion only for selected feature points.

Sparse Optical Flow:

- Tracks selected important points.
- Requires less computation.
- Produces fewer motion vectors.
- Example: Lucas-Kanade.

Dense Optical Flow:

- Estimates motion throughout the image.
- Requires more computation.
- Produces a detailed motion field.
- Example: Farneback.

Therefore, Sparse Optical Flow is useful for efficient feature tracking, while Dense Optical Flow is useful for detailed motion analysis.

Q4. Compare the Lucas-Kanade and Farneback optical flow algorithms.

Lucas-Kanade is a Sparse Optical Flow algorithm that tracks selected feature points between frames. It is relatively computationally efficient and produces motion vectors only at the selected points.

Farneback is a Dense Optical Flow algorithm that estimates motion across the image. It provides a more complete representation of the motion field but generally requires more computational resources.

Lucas-Kanade is suitable for feature tracking and object tracking, whereas Farneback is suitable for detailed motion analysis and dynamic scene understanding.

In this experiment, Lucas-Kanade was visualized using motion arrows and Farneback was visualized using a color-coded motion representation.

Q5. What assumptions are made while computing optical flow?

The major assumptions used in optical flow estimation are:

1. Brightness constancy: The intensity of a moving point remains approximately constant between consecutive frames.
2. Small motion: The displacement between consecutive frames is assumed to be relatively small.
3. Spatial coherence: Neighboring pixels generally have similar motion.
4. Objects maintain their appearance: The visual properties of an object are assumed not to change significantly between frames.
5. Motion is relatively smooth: Motion is assumed to vary smoothly over nearby pixels or regions.

Q6. What factors can affect the accuracy of optical flow estimation?

Several factors can affect optical flow accuracy:

1. Object speed: Very fast movement can cause large displacement between frames and make tracking difficult.
2. Lighting changes: Changes in illumination can violate the brightness constancy assumption.
3. Camera motion: Camera movement can produce optical flow across the entire scene.
4. Image texture: Regions with very little texture may not contain sufficient features for reliable tracking.
5. Occlusion: When an object becomes hidden by another object, its motion may be difficult to estimate.
6. Motion blur: Fast movement can blur image features and reduce tracking accuracy.
7. Frame rate: A low frame rate can create larger displacement between consecutive frames.
8. Noise and image quality: Poor-quality or noisy images can affect feature detection and motion estimation.

Q7. Mention five real-world applications of optical flow.

Five important applications of optical flow are:

1. Autonomous driving – detecting movement of vehicles, pedestrians, and other objects.
2. Video surveillance – detecting and analyzing movement in surveillance videos.
3. Object tracking – tracking objects across consecutive video frames.
4. Robotics – helping robots understand movement in their surroundings.
5. Gesture and action recognition – analyzing human movements and gestures.

Other applications include sports analytics, augmented reality, motion detection, and video stabilization.

Q8. Why are grayscale images generally used for optical flow computation?

Grayscale images are generally used because optical flow primarily depends on changes in image intensity and image gradients.

A grayscale image contains one intensity channel instead of three color channels. Therefore, it reduces computational requirements and simplifies gradient calculations.

Using grayscale also makes the optical flow calculation faster while retaining the important brightness information required for estimating motion.

Q9. What are the limitations of optical flow algorithms in real-world environments?

The main limitations of optical flow algorithms include:

1. Sensitivity to lighting changes.
2. Difficulty handling very fast motion.
3. Problems caused by occlusion.
4. Motion blur can reduce accuracy.
5. Textureless regions may provide insufficient information.
6. Camera movement can produce unwanted global motion.
7. Large displacement between frames can make tracking difficult.
8. Dense optical flow can require significant computational resources.
9. Noise and low-quality video can affect motion estimation.
10. Complex scenes containing multiple moving objects can make interpretation difficult.

Q9. What are the limitations of optical flow algorithms in real-world environments?

The main limitations of optical flow algorithms include:

1. Sensitivity to lighting changes.
2. Difficulty handling very fast motion.

3. Problems caused by occlusion.
4. Motion blur can reduce accuracy.
5. Textureless regions may provide insufficient information.
6. Camera movement can produce unwanted global motion.
7. Large displacement between frames can make tracking difficult.
8. Dense optical flow can require significant computational resources.
9. Noise and low-quality video can affect motion estimation.
10. Complex scenes containing multiple moving objects can make interpretation difficult.

Q10. How does optical flow contribute to autonomous driving, video surveillance, and action recognition?

Optical flow provides information about the movement of objects and surfaces in video sequences.

In autonomous driving, optical flow can help analyze the movement of vehicles, pedestrians, and other objects and can contribute to understanding the dynamic environment.

In video surveillance, optical flow can be used to detect and track moving objects and identify changes in movement within a monitored scene.

In action recognition, optical flow captures patterns of human movement across consecutive frames. These motion patterns can be used along with other visual information to recognize activities and gestures.